

Savitribai Phule Pune University

Modern Education Society's Wadia College of Engineering, Pune

19, Bund Garden, V.K. Joag Path, Pune – 411001.

ACCREDITED BY NBA AND NAAC WITH 'A++' GRADE

DEPARTMENT OF COMPUTER ENGINEERING



A

MINI PROJECT REPORT

ON

“TweetLens: Explainable Multi-Model Sentiment Analysis of Tweets”

B.E. (COMPUTER)

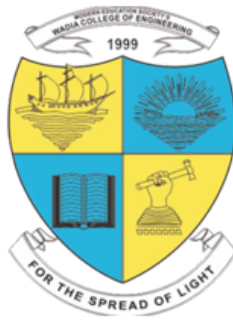
SUBMITTED BY

Yashvant Dayanand Mane

F24121004

(Academic Year: 2026-2027)

DEPARTMENT OF COMPUTER ENGINEERING
MODERN EDUCATION SOCIETY'S WADIA COLLEGE OF ENGINEERING, PUNE



CERTIFICATE

This is to certify that the “**Mini project report**” entitled “**TweetLens: Explainable Multi-Model Sentiment Analysis of Tweets**” submitted by **Yashvant Dayanand Mane** and **PRN is F24121004** is work done by him and submitted during 2026-27 academic year, in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF ENGINEERING in COMPUTER ENGINEERING**, at **MESWCOE**

Date :

Head of the Department

Place :

of Computer Engineering

Sign :

ACKNOWLEDGEMENT

I express my sincere thanks to the Head of the Department of Computer Engineering and to all the faculty members of Modern Education Society's Wadia College of Engineering, Pune, for their support, guidance and encouragement throughout this mini project. I am grateful to the department for providing the facilities and the learning environment that made this work possible.

I also thank the researchers who released the TweetEval benchmark and the open-source communities behind scikit-learn, FastAPI and React, whose work this project builds on. Finally, I thank my family and friends for their constant support.

Yashvant Dayanand Mane

PRN: F24121004

ABSTRACT

Social media posts such as tweets carry public opinion about products, services and events, but they are short, informal and full of hashtags, emojis, slang, negation, sarcasm and code-mixed Hinglish, which makes automatic sentiment analysis difficult. This project presents **TweetLens**, a full-stack web application that classifies a tweet as negative, neutral or positive using four machine learning models: a calibrated VADER lexicon model, Multinomial Naive Bayes, Logistic Regression on word and character TF-IDF features, and a Multi-Layer Perceptron neural network.

A nine-stage preprocessing pipeline designed for tweets (emoji-to-word conversion, hashtag splitting, contraction expansion, Hinglish translation, negation fusion and stopword removal) prepares the text. The models are trained on the TweetEval sentiment benchmark (45,615 tweets) and tested on 12,284 held-out tweets, where Logistic Regression performs best with 62.4% accuracy and 0.623 macro F1. The application explains each prediction word by word, compares the four models side by side, tests them on deliberately difficult tweets, streams tweets live and analyses whole CSV files. It is deployed on Render and Netlify.

Keywords: sentiment analysis, Twitter, natural language processing, TF-IDF, logistic regression, Naive Bayes, explainable AI.

1. Mini Project Title

TweetLens: Explainable Multi-Model Sentiment Analysis of Tweets. A machine learning web application that reads the sentiment of a tweet and shows it like litmus paper: the strip turns red for negative, violet for neutral and blue for positive, with a score from -1 to $+1$.

2. Problem Statement

Millions of tweets are posted every day, and organisations need to know what people feel about their products and services. Reading them manually is impossible at that scale, so sentiment must be analysed automatically. Tweets are harder than ordinary text: they are at most 280 characters long and contain hashtags (#NeverAgain), @mentions, links, emojis, stretched words ("soooo goood"), negation ("not bad at all"), sarcasm and, in India, Hinglish ("movie ekdum bakwas thi").

The problem is to *build and evaluate machine learning models that classify a tweet as negative, neutral or positive, explain their predictions, and test where they fail*, delivered as an interactive web application that can analyse single tweets, compare models and process whole files of tweets.

3. Objectives

1. To design a preprocessing pipeline for tweets that handles links, emojis, hashtags, contractions, Hinglish words, negation and stopwords.
2. To train and compare four classifiers: VADER, Multinomial Naive Bayes, Logistic Regression and a neural network.
3. To evaluate the models on a held-out benchmark with accuracy, macro F1, macro recall and confusion matrices.
4. To explain each prediction by showing how much every word moved the score.
5. To detect emotion (anger, joy, optimism, sadness) and irony in tweets.
6. To test robustness on hand-written difficult tweets (negation, sarcasm, Hinglish, emoji-only, slang).
7. To build a responsive web interface and deploy the system to the cloud.

4. Hardware and Software Requirements

4.1 Hardware Requirements

Component	Requirement
Processor	Dual-core 64-bit CPU or better (development: 4-core CPU)
RAM	4 GB minimum; the running server uses about 220 MB
Storage	2 GB free for dependencies, dataset and trained models
GPU	Not required; all models train on a CPU in about 2–3 minutes

4.2 Software Requirements

Category	Software / Library
Operating system	Linux, Windows or macOS
Backend language	Python 3.11
Machine learning	scikit-learn 1.9, NumPy 2.4, pandas 3.0, joblib
NLP	vaderSentiment 3.3, emoji 2.16, wordsegment 1.3
Web API	FastAPI 0.142, Uvicorn 0.54
Frontend	TypeScript, React 19, Vite 8, D3, framer-motion
Testing and tools	pytest, Vitest, Playwright, Git, GitHub, VS Code
Deployment	Render (API) and Netlify (web app)

5. Dataset Description

The main dataset is the **TweetEval sentiment benchmark** (Barbieri et al., 2020), taken from SemEval-2017 Task 4A. Each tweet is labelled by human annotators as 0 = negative, 1 = neutral or 2 = positive, and usernames are anonymised as @user. It has fixed splits, so results are comparable with published work.

Split	Tweets	Negative	Neutral	Positive
Train	45,615	15.5%	45.3%	39.1%
Validation	2,000	15.6%	43.4%	40.9%
Test	12,284	32.3%	48.3%	19.3%

Table 1: TweetEval sentiment splits and class distribution

The test split has twice the share of negative tweets that the training split has, so the models use balanced class weights and macro-averaged metrics are reported. Additional data: TweetEval Emotion (3,257 training tweets) and Irony (2,862 training tweets) for the emotion and irony readers, 1,200 dated tweets from the Twitter US Airline Sentiment dataset (February 2015) for bulk testing, and 36 hand-written difficult tweets for the robustness test.

6. Methodology / Algorithm

6.1 System architecture

```
Raw tweet -> 9-stage preprocessing -> counts / TF-IDF features
                                         -> Naive Bayes, Logistic Regression, Neural net,
                                         Emotion reader, Irony reader
Raw tweet -> VADER lexicon scores -> calibration -> VADER

Each model -> P(negative), P(neutral), P(positive)
             -> score = P(positive) - P(negative) -> label + word-level explanation
```

The backend (Python, FastAPI) holds the pipeline, the trained models and a REST API; the frontend (React, TypeScript) calls it. Models are trained once by a script and saved with joblib.

6.2 Preprocessing algorithm (9 stages)

1. Tokenise, keeping URLs, @mentions, #hashtags, emojis and emoticons whole.
2. Remove links, mentions and retweet marks.
3. Convert emojis and emoticons to words: 😡 → enraged_face, :(→ frowning_face.
4. Split hashtags: #NeverAgain → never again.
5. Lowercase and squeeze stretched words: SOOOO → soo.
6. Expand contractions: don't → do not.
7. Translate Hinglish words (58-word lexicon): bakwas → rubbish, achha → good, nahi → not.
8. Fuse negation: not happy → NOT_happy; Hindi "nahi" fuses backwards: achhi nahi → NOT_good.
9. Remove stopwords and most punctuation (keeping ! and ?).

6.3 Features and models

Model	Algorithm and features	Key settings
VADER	Lexicon of about 7,500 rated words with rules for negation, capitals and "but"; a small logistic regression turns its scores into probabilities	Balanced weights
Naive Bayes	Multinomial NB on word unigram and bigram counts: $P(\text{class} \text{words}) \propto P(\text{class}) \cdot \prod P(\text{word} \text{class})$	$\alpha = 1.0$
Logistic Regression	Softmax regression on TF-IDF of word 1–2-grams plus character 2–5-grams (up to 300,000 features)	$C = 1.0$, balanced weights
Neural net (MLP)	One hidden layer of 128 ReLU units on word TF-IDF, Adam optimiser	$L2 = 0.01$, early stopping

TF-IDF weight: $(1 + \ln tf) \times (\ln((1 + N) / (1 + df)) + 1)$, where N is the number of tweets and df the number of tweets containing the term. Hyperparameters were chosen on the validation split.

6.4 Scoring, explanation and evaluation

The score is $P(\text{positive}) - P(\text{negative})$, from -1 to $+1$, and the label is the most probable class. Each word's influence is found by **leave-one-out**: the word is removed, the tweet is scored again, and the change in score is its weight. Models are compared by accuracy, per-class precision, recall and F1 ($F1 = 2PR / (P + R)$), macro F1 and macro recall (the mean over the three classes), and confusion matrices.

7. Practical Implementation

7.1 Training code (Logistic Regression)

```
word = TfidfVectorizer(token_pattern=r"\S+", ngram_range=(1, 2),
                        min_df=2, sublinear_tf=True)
char = TfidfVectorizer(analyzer="char_wb", ngram_range=(2, 5),
                        min_df=3, max_features=300_000)
clf = LogisticRegression(C=1.0, max_iter=3000, class_weight="balanced")

logreg = Pipeline([("features", FeatureUnion([("word", word), ("char", char)])),
                  ("clf", clf)])
logreg.fit(clean_many(x_train), y_train) # clean_many = 9-stage pipeline
joblib.dump(logreg, "artifacts/logreg.joblib")
```

7.2 Training output

```
Reading TweetEval sentiment: pipeline over 59,899 tweets in 13s
emotion: macro F1 0.684      irony: macro F1 0.687
VADER          accuracy 0.554 macro F1 0.550 macro recall 0.582
Naive Bayes    accuracy 0.591 macro F1 0.585 macro recall 0.595
Logistic regression accuracy 0.624 macro F1 0.623 macro recall 0.635
Neural net     accuracy 0.560 macro F1 0.564 macro recall 0.589
Done in 129s.
```

7.3 Results on the test set (12,284 tweets)

Model	Accuracy	Macro F1	Macro recall	Time / tweet
VADER (calibrated)	0.554	0.550	0.582	0.17 ms
Naive Bayes	0.591	0.585	0.595	1.02 ms
Logistic Regression	0.624	0.623	0.635	1.93 ms
Neural net (MLP)	0.560	0.564	0.589	0.87 ms
<i>Baseline: always "neutral"</i>	0.483	0.217	0.333	–

Table 2: Test-set performance. Logistic Regression is the best model and the default in the app.

True \ Predicted	neg	neu	pos
negative	2,728	1,013	231
neutral	1,687	3,416	834
positive	211	639	1,525

Table 3: Confusion matrix of Logistic Regression

Robustness test (36 tweets)	Correct
VADER	15
Naive Bayes	15
Logistic Regression	20
Neural net	19

Table 4: Hand-written difficult tweets

Logistic regression reaches 63.5% macro recall, slightly above the 62.9% reported for the SVM and FastText baselines in the TweetEval paper. Most errors are between neutral and its neighbours; negative and positive are rarely confused. In the robustness test, the Hinglish stage lets the trained models answer all four Hinglish tweets correctly, while every model misreads all four sarcastic tweets.

7.4 Application screenshots

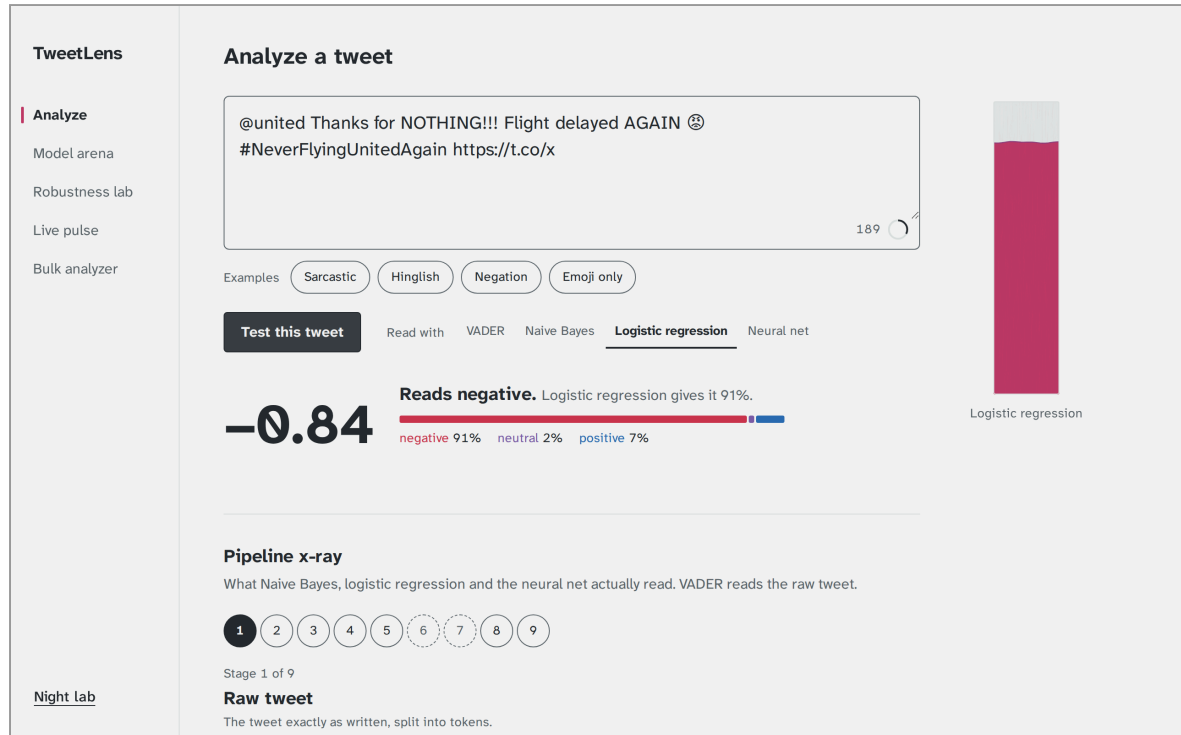


Figure 1: Analyze page. A complaint tweet scores -0.84 (91% negative) and the litmus strip turns red.

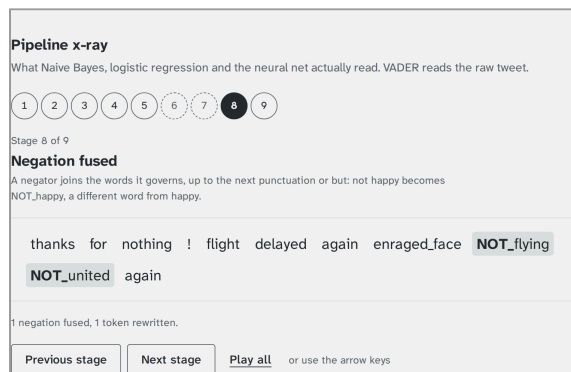


Figure 2: Pipeline X-Ray, negation fused into NOT_ words.

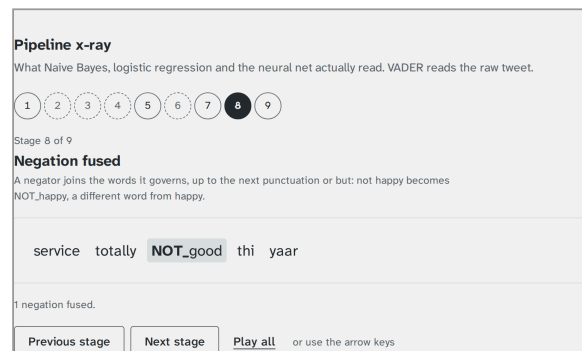


Figure 3: Hinglish "achhi nahi" becomes NOT_good.

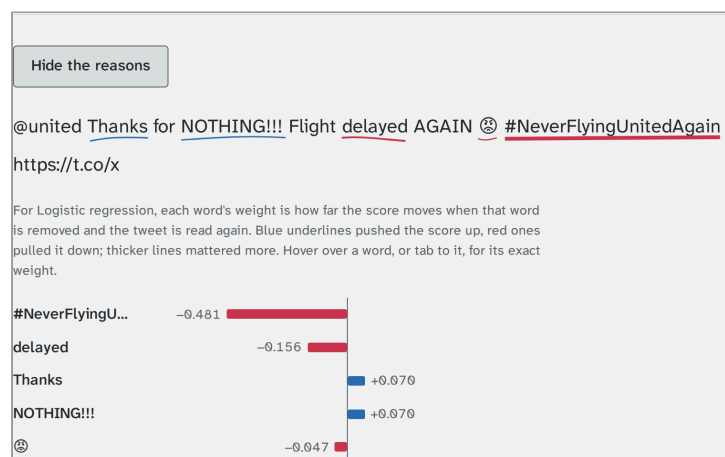


Figure 4: "Why this prediction?" Red words pulled the score down, blue pushed it up, with exact weights.

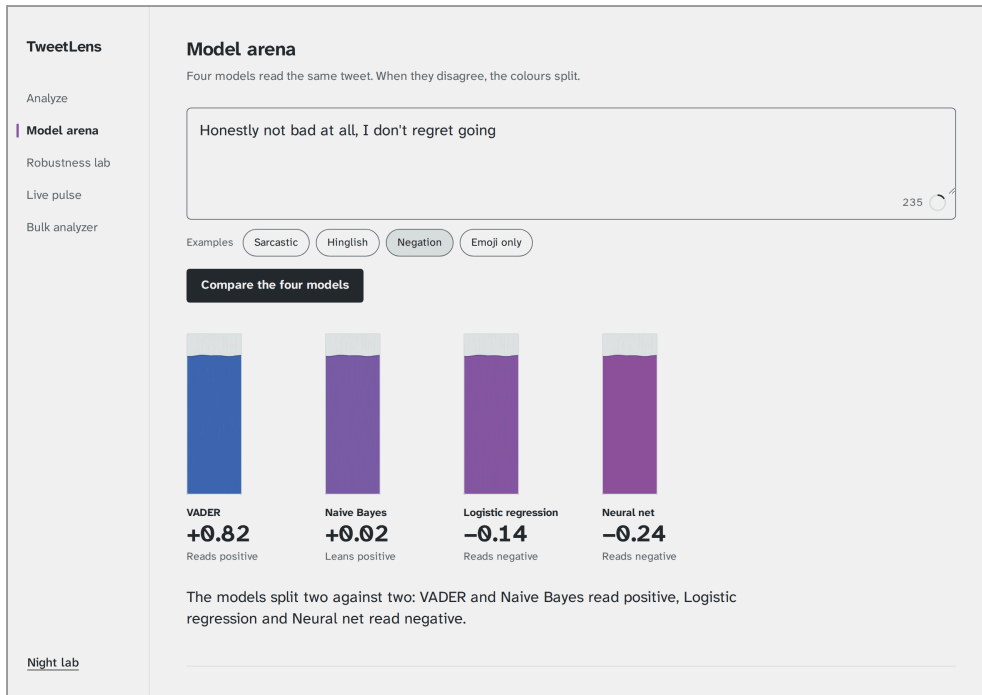


Figure 5: Model Arena. Four models read the same tweet; here they split two against two.

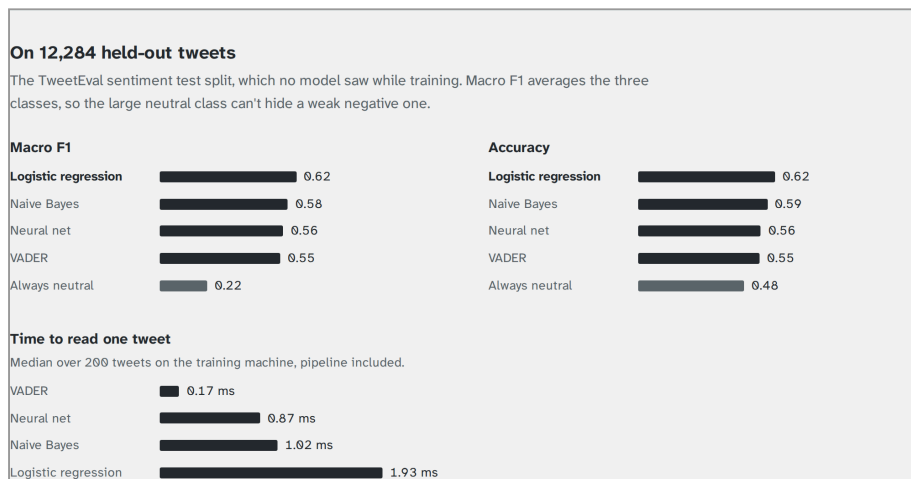


Figure 6: Benchmark of the four models on 12,284 held-out tweets.

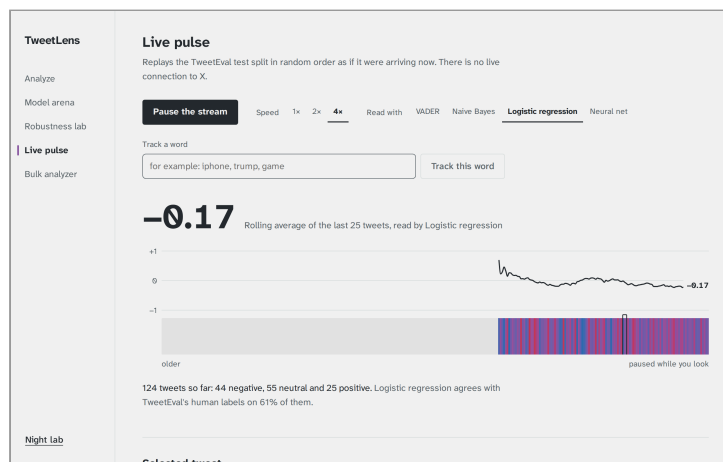


Figure 7: Live Pulse. Each band is one tweet; the line is the rolling average of the last 25.

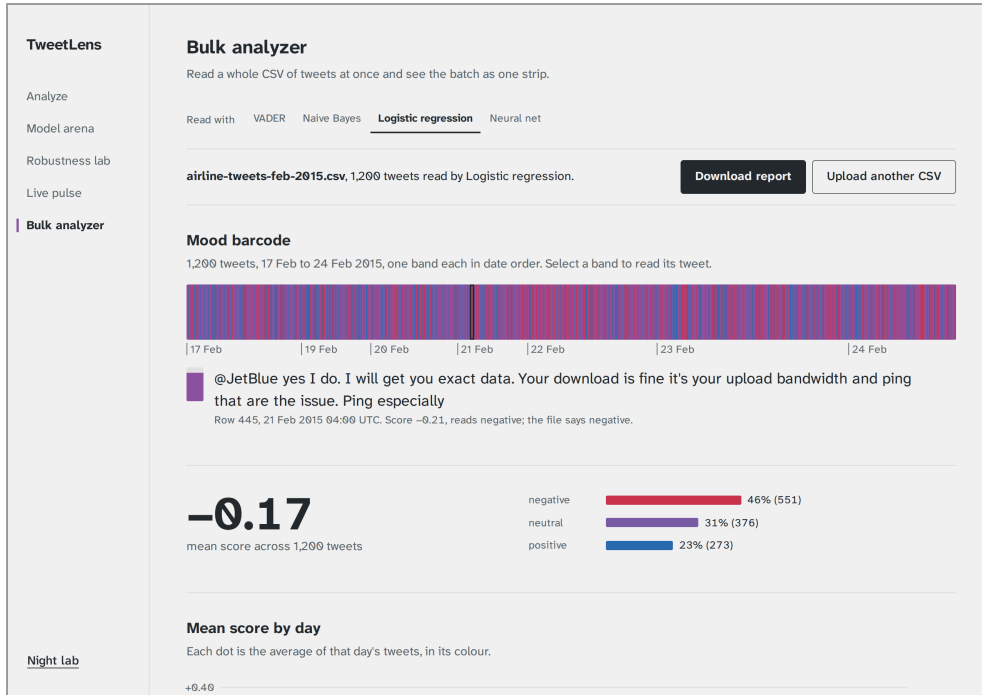


Figure 8: Bulk Analyzer on 1,200 airline tweets: the mood barcode (one band per tweet in date order) and class shares. Against the file's labels, accuracy is 0.69.

8. Conclusion

TweetLens shows that classical machine learning, combined with preprocessing designed for tweets, gives a fast and explainable sentiment analyser. **TF-IDF with Logistic Regression performed best** (accuracy 62.4%, macro F1 0.623, macro recall 63.5%), slightly above published bag-of-words baselines on the same benchmark, while answering in about 2 ms per tweet. Hashtag splitting and the Hinglish stage clearly helped on difficult tweets, but sarcasm defeated every model, so the app shows a separate irony warning. Word-level explanations, the step-by-step pipeline view and side-by-side model comparison make the system transparent rather than a black box.

Future scope: fine-tuning a transformer such as Twitter-RoBERTa, a larger Hinglish vocabulary, better sarcasm detection and a real Twitter/X data stream.

References

1. F. Barbieri et al., "TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification," Findings of EMNLP, 2020.
2. S. Rosenthal, N. Farra, P. Nakov, "SemEval-2017 Task 4: Sentiment Analysis in Twitter," SemEval, 2017.
3. C. J. Hutto, E. Gilbert, "VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text," ICWSM, 2014.
4. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," JMLR 12, 2011.